Infrastructure and Onboarding

QUANTT Credit Trading · technical reference

Version 1.0 · 16 August 2026

github.com/quanttqueensu/CreditTrading

CONTENTS

1. Overview

We operate a systematic credit strategy on a $500,000 Interactive Brokers paper account. The strategy trades closed-end fund discounts, holds equal dollars long and short, and targets 6% annualised volatility. Five scheduled jobs run the book unattended on weekdays, and seven preflight checks gate every trade.

This document is our technical reference. Part 2 covers onboarding for new members. Parts 3 to 7 are reference material. Part 8 lists open issues and is maintained as a live list. Each part is self-contained so sections can be revised independently through the year. Any change to the system requires a changelog entry below.

DATE	VERSION	CHANGE	AUTHOR
2026-08-16	1.0	First issue, reflecting the state of the system at the end of the summer	S. Jarvis

2. Onboarding

2.1 Environment

Python 3.11 or later. The trading machine runs 3.13.5 under Anaconda.

```
git clone https://github.com/quanttqueensu/CreditTrading.git
cd CreditTrading
python3 -m pip install -r requirements.txt
```

The `data` directory is 3.8 GB and is excluded from the repository. Section 2.3 covers reconstruction. Broker access is needed only for trading and is limited to two or three members.

2.2 Required reading

ORDER	DOCUMENT	CONTENT
1	`HOW_WE_GOT_HERE.md`	Chronological account of the summer, including failed approaches. No finance background assumed.
2	`RESEARCH_AND_METHODOLOGY.md`	Our criteria for establishing that a result is real.
3	`RESEARCH_STATE.md`	Live project state: deployed, killed, queued. Read first and written last in every research session.

ORDER	DOCUMENT	CONTENT
4	`results/AUDIT_2026-07-31.md`	End-to-end audit identifying three defects in the deployed configuration.
5	`ops/AUTOMATION.md`	Automation runbook.

2.3 Reconstructing the data directory

Most sources are free and can be refetched in this order:

```
python3 scripts/cef/stage_cef.py              # closed-end fund prices and NAVs
python3 scripts/fetch/refresh_market_feeds.py # VIX complex, Treasury futures
python3 scripts/fetch/fetch_live_sources.py   # daily free sources
python3 scripts/holdings/ingest_holdings.py   # ETF holdings, ~11k bonds
python3 scripts/holdings/fetch_nav_multi.py   # issuer-published NAVs
```

Two datasets cannot be reconstructed and must be copied from the team lead. The first is the daily ETF holdings history: issuers publish only the current day and silently ignore any date parameter, so our record begins 29 July 2026 and extends forward one day at a time. Missed days are unrecoverable, which is why collection runs as a job separate from trading. The second is the TRACE bond data and forced-flow panels, 3.3 GB in `data/forced_flow2`, which came from a licensed source.

2.4 Verification

New members run the validation battery before anything else:

```
python3 scripts/cef/validate.py
```

Expected output is a gross Sharpe near 1.26 and a net Sharpe near 0.82. Any deviation is treated as a finding and reported to the team rather than assumed to be local error.

2.5 Working standards

A false positive is the worst outcome of a research session, worse than a null result. Promising results are attacked before they are reported.

Each session adds a new data source rather than a new parameter to an existing model. Repeated search over the same data yields diminishing returns and raises the statistical threshold for all subsequent work.

Every test is counted. `RESEARCH_STATE.md` holds a running total that never resets. At 10 trials the best pure-noise result scores approximately 2.1; at 162 trials, approximately 3.2. Results are assessed against the running total.

Every failure is classified into one of the seven categories defined in the methodology document. “It did not work” is not an acceptable entry.

Shutdown rules are written before deployment, not after.

3. Strategy

3.1 Mechanism

A closed-end fund trades on an exchange but issues its shares once, after which the share count is fixed. It publishes the value of its holdings daily as the NAV. The share price is set independently by the exchange, and the two diverge.

An ordinary ETF has authorised participants who exchange shares for the underlying bonds and back, closing any gap within minutes. We measured this mechanism operating: the gap on high yield ETFs compressed from 188 basis points in 2008 to 3.8 by 2026, and that compression is what killed our two earlier strategies. Closed-end funds have no equivalent mechanism. Credit closed-end funds trade at a mean discount of 3.16% with a standard deviation of 5.95%, roughly 150 times the ETF gap.

3.2 Implementation

For each fund we compute the discount, z-score it against that fund’s own trailing 252 trading days, and rank the cross-section. We hold the cheapest funds long and the richest short in equal dollar amounts, then scale the book to a 6% annualised volatility target. Scoring each fund against its own history rather than against peers is deliberate, since several funds carry a permanent structural discount that carries no information.

3.3 Evidence

The test that killed the ETF version reverses here. For an ETF the discount predicts the NAV, with t-statistics of $+15$ to $+24$, meaning a lagging valuation converges to a price that was already correct; trading it means opposing genuine price discovery. For closed-end funds the discount predicts the price, with a mean t of -1.75 and 6 of 18 funds beyond -2, and predicts the NAV only weakly.

Against high yield, investment grade, rates, equity and volatility, the R-squared is 0.005 and alpha is $+2.68\%$ per year at $t = 3.11$.

We then attempted twice to invalidate the result. The live book is 66% net short municipal funds and 57% net long taxable funds, so the natural hypothesis was a disguised sector position, which is what one earlier candidate proved to be.

CONTROL SET	ALPHA P.A.	T	R^2
Base factor set	+8.21%	+5.67	0.0037
Plus municipal ETFs	+8.88%	+5.69	0.0117
Plus municipal ETFs and duration spread	+8.83%	+5.67	0.0125
All five closed-end fund group factors	+7.63%	+5.90	0.0057

The final row is the decisive control, since municipal closed-end fund discounts do not track municipal ETFs. Under it every group beta is at or below 0.025 and the alpha is unchanged.

3.4 Live configuration

Frozen spec: `ops/specs/cef_discount.frozen.json`, id `cef_discount.v5.20260731`.

PARAMETER	VALUE	NOTE
Universe	17 funds	AWF BIT DSL HYT JFR MHD MQY NAD NEA NVG NZF PCN PDI PDO PFN PHK PTY
z-window	252 days	Reverted from 63 after the sealed holdout failed that change
Rebalance	2 days	Reduced from 5; 2 is the measured optimum
Volatility target	6% p.a.	
Minimum ADV	$3,000,000	
Maximum NAV age	3 business days	A stale NAV produces a blind signal, not a cheap fund
Minimum names	6	
Order type	Market-on-close	See section 5.5
Capital	$500,000	
Gross cap	$1,300,000	

Every modified parameter carries a note field in the spec recording the measurement that justified it. No value is changed without one.

3.5 Declared weaknesses

All of the following were recorded before capital was committed and remain true.

The honest net Sharpe of the deployed configuration is 0.51, not 0.82. The backtest entered at day *t*'s close using day *t*'s NAV, which publishes after that close; a market-on-close order fills at *t+1*'s close. The correction costs 38% of the Sharpe.

Kurtosis is 41.2, so sharp single-day losses are expected. Recent performance is flat: the most recent walk-forward block scored 0.05 and the 2023 to 2026 net Sharpe is 0.30.

The strategy performs better in calm markets than dislocated ones, which is the opposite of our initial hypothesis. Net Sharpe by dispersion quintile runs 1.24, 0.59, 0.80, −0.23, 0.68 from calmest to most dislocated. Sizing up into dislocation produced a 31.5% drawdown in 2008.

The spec contains no group exposure limit. The 66% municipal tilt does not register as a return risk in the regressions but is unmonitored.

Distribution data is fetched and unused. `data/cef/cef_dist_features.parquet` holds 11,988 rows flagging cuts and raises. A distribution cut re-rates a discount permanently wider, which is the standard loss mode for this strategy, and no defence exists. Our event study found cuts move the discount against the obvious trade, so this is a risk control problem rather than a signal. It remains open.

3.6 Kill rule

Committed before deployment and reviewed at 60 live sessions, not before. The strategy is killed if live net Sharpe falls below zero, if realised slippage exceeds twice modelled for five consecutive sessions, or if the top-minus-bottom discount spread falls below 12%, half its current level.

Automatic enforcement is currently disabled. Sleeve kills return OK and log a warning, and book drawdown suspension is set to 99%. The paper deployment exists to generate evidence, and a strategy that suspends itself stops producing the data it was deployed to collect; with no capital at risk the usual reason to cut a losing book does not apply. The kill rule is therefore a checklist applied by a human at session 60. Broker margin limits remain in force.

4. Code

4.1 Layout

```
src/deploy/          live trading framework
  sleeve.py          the contract every strategy implements
  registry.py        strategy type to class mapping, spec validation
  portfolio.py       consolidates sub-ledgers into a book view
  run_book.py        session entry point
  exec_ledger.py     sub-ledgers strategies trade through
  fills.py           fill price model: half spread, market impact
  risk.py            per-strategy kill and halve switches, book limits
  report.py          daily reports
  sleeves/           cef_discount, null_trader, credit_rv, static_weights
  broker/            base interface, ibkr.py, simulator.py
  lib/               v2 namespace, additive, does not modify v1
src/backtest/        daily engine, lookahead guard, walk-forward
src/strategies/      credit relative value research code
src/data/            Cloudflare R2 access for the WRDS mirror
ops/                 preflight, halts, ledgers, monitoring, reports
scripts/             research and data scripts by family
config/              cost models and credentials
results/             outputs, one directory per research family
docs/                this document, project intro, summer summary
data/                3.8 GB, excluded from git
```

4.2 Strategy contract

Every strategy implements four methods defined in `src/deploy/sleeve.py`: `instruments()`, `history_warmup_trading_days()`, `target_positions()`, and `risk_check()`, which returns OK, HALVE or KILL. A strategy is registered by decorator with an `alloc_type` name, after which the registry validates any spec claiming that type. We added this after shipping two defects in which a spec type was accepted and validated with no implementing class behind it, failing only at run time.

4.3 The deployed strategy

`src/deploy/sleeves/cef_discount.py`, 239 lines. It computes the discount as `100 * (price - nav) / nav`, z-scores it against a rolling 252-day window shifted by one day, and clips at ±4. It then applies the rebalance gate, anchored to the trading-day index so the schedule does not drift, drops names failing the ADV, NAV age and minimum weight filters, neutralises the book, and scales to the volatility target using 63-day trailing realised volatility with the scalar clipped to [0.2, 2.5].

The minimum weight filter runs before neutralisation. It previously ran after, leaving the book 0.37% net short, which is precisely the exposure the strategy exists to avoid. Residual is now 1e-6.

5. Live trading system

5.1 Broker interface

Interactive Brokers TWS, paper account DUQ199038, at 127.0.0.1:7497.

The account is denominated in Canadian dollars, so all US dollar sizing converts. Net liquidation of 1,000,674 CAD was 714,059 USD at the 31 July rate.

We hold no real-time data subscription; quotes arrive 15 minutes delayed. Cost estimates therefore come from historical measurement rather than live quotes. Historical bid-ask data is available under a separate entitlement and is how we measure execution cost.

We use `ib_async`, not `ib_insync`. The latter is unmaintained and hangs indefinitely in its asyncio handshake on Python 3.12 and above; TWS answers a raw socket normally while the library never returns, which presents identically to a dead gateway. `src/deploy/broker/ibkr.py` imports `ib_async` first and falls back only if absent.

TWS restarts daily and requires an interactive login, which is currently our largest operational weakness.

`orderRef` does not survive the round trip on this TWS build; every execution returns an empty ref. Fill attribution is therefore by ticker, which is safe only while no two deployed strategies trade the same symbol. `ops/capture_fills.py` refuses to run if that condition is violated.

Credentials live in `config/.env`, excluded from git:

```
R2_ACCESS_KEY_ID / R2_SECRET_ACCESS_KEY / R2_ENDPOINT / R2_BUCKET
IBKR_HOST=127.0.0.1 / IBKR_PORT=7497 / IBKR_CLIENT_ID=17
```

Each scheduled job overrides the client id to prevent collisions. The CEF job uses 45; audit scripts use 93 and 95.

5.2 Books and ledgers

BOOK	STRATEGIES	CAPITAL	STATUS
cef_discount_paper	cef_discount	$500,000	Live
phase0_null	null_trader	$640,000	Live control experiment
benchmarks_paper	five static weight books	$20,000 each	Live reference

BOOK	STRATEGIES	CAPITAL	STATUS
credit_rv_paper_v1	credit_rv	$1,000,000	Killed, retained for reference

Each live book maintains a shadow ledger: `nav.csv` (daily value, cash, cost, turnover, return), `positions.csv`, `orders.csv`, `trades.csv` (modelled fills with cost breakdown), `broker_fills.csv` (real executions with exec id and commission), `slippage.csv` (realised against modelled), and `manifest.json` (row counts and last dates, verified on load).

The shadow ledger books modelled fills by design and remains the P&L source; real executions are held separately. The exception is the null trader, rebuilt from real fills because measuring real execution is its purpose.

5.3 The arm() gate

This is the primary safety control. It runs after strategy registration and before any order, and divides authority: the broker is authoritative for how many shares exist, the ledger for which strategy owns them.

It adopts quantities from the broker's actual positions, consults sibling book specs so a symbol another book also trades is never taken from the account net, and refuses to arm where attribution is ambiguous, in which case `place_targets` raises `NotArmed` rather than transmitting.

The non-obvious case: the account held 823 shares of HYG, of which the null trader owned 541 and the benchmark books 282. Adopting the account net would have sold 282 shares the strategy never bought.

This control exists because on 31 July the ledger froze at its funding row showing zero positions while the account held 35 positions worth $2.07M gross. The following session would have re-bought both books in full.

5.4 Manual operation

```
# dry run: compute targets, log, transmit nothing
python3 -m src.deploy.run_book --asof YYYY-MM-DD \
    --book ops/books/cef_discount_book.json --source yfinance --dry-run

# live paper session
python3 -m src.deploy.run_book --asof YYYY-MM-DD \
    --book ops/books/cef_discount_book.json --source yfinance
```

Re-running an armed book is not idempotent; each run stacks an additional order set with no deduplication. On the evening of 31 July four armed runs stacked 79 unseen market-on-close orders, invisible because `openTrades()` returns only the querying client's orders, and

`cancelOrder` failed across client ids with error 10147. `reqGlobalCancel()` cleared them. Pending orders must be cancelled before any manual re-run.

5.5 Execution

The strategy decides in the evening because its signal requires the NAV, which publishes after the close. On the first live day plain market orders rested overnight and filled at 07:27 ET, two hours before the exchange opened, in funds trading $3M to $45M per day with negligible pre-market depth.

TRADED	SLIPPAGE	REALISED	MODELLED	RATIO
$682,351	$6,405	0.94%	~0.10%	9.4×

Worst names were BIT at 2.87%, DSL at 2.62% and PFN at 2.27%. Slippage ran against us on every buy and every sell, the signature of crossing a wide spread rather than noise. The three most liquid municipal funds filled at approximately 0.00%, consistent with their being the only names with real pre-market depth.

Priced at decision prices the book was up $50; priced at fills it was down $7,350. At 24 rebalances per year, 0.94% per rebalance is 22.6% annually against a strategy earning 4.85%.

We excluded the alternative explanation by reconciling all 17 funds against the broker's daily bars over 10 days: median disagreement 0.000%. Our prices are exact, so this is purely execution.

The remedy is market-on-close orders, which execute in the closing auction, the deepest and tightest liquidity of the day and the point the backtest assumed. Routing was verified live at 16:39 ET, after the exchange cutoff and after the close, matching the conditions the 17:15 job encounters; it returned `PreSubmitted` with no error and cancelled cleanly.

Whether this remedy is sufficient is the most important open question in the project, ahead of returns. We cannot slow the strategy to escape the cost: net Sharpe by holding period runs 0.62 at one day, 0.73 at two, 0.51 at five, 0.30 at ten and 0.20 at twenty-one. If slippage remains above twice modelled, these instruments are too expensive to trade at the only frequency at which the edge exists.

6. Automation

6.1 Scheduled jobs

JOB	SCHEDULE	FUNCTION
com.quantt.phase0.daily	09:35 Mon–Fri	Null trader control experiment
com.quantt.cef.daily	17:15 Mon–Fri	CEF strategy, MOC for the next close

JOB	SCHEDULE	FUNCTION
com.quantt.collect.daily	18:30 Mon–Fri	Data collection
com.quantt.watchdog.daily	19:30 Mon–Fri	Alerts on any job that did not run
com.quantt.weekly	Sat 09:00	Book roll-up report

All five execute a single file, `~/Library/Application Support/quantt/launch_job.py`.

6.2 The TCC constraint

No scheduled job may run through the shell. The repository sits under `~/Desktop`, which macOS protects with TCC. A launchd agent holds no Full Disk Access, so `/bin/bash` cannot read a script located there. The 09:35 job on 31 July exited 126, transmitted nothing, and wrote no log.

The Anaconda interpreter holds Full Disk Access; the system shell does not. The entry point is therefore a Python file located outside the protected directory.

This failure mode is invisible under manual testing, because a Terminal holds Full Disk Access. Scheduled jobs must be tested with `launchctl kickstart`, never by loading fresh: a freshly loaded agent inherits the loading Terminal's permissions and passes a test the real scheduled run fails.

Consequently `ops/schedule/run_cef.sh` and its siblings are not what runs daily. They are retained for manual use only.

6.3 Session structure

```
1. REFRESH    fetch prices and NAV        fail -> no trading, still logs
2. PREFLIGHT  assess trading safety       fail -> no trading, still logs
3. TRADE      run the book                fail -> halt and alert
4. CAPTURE    record real executions      always runs, even after 1-3 fail
```

Trading is dangerous when state is wrong and fails closed. Data collection is lost only if it does not occur, since issuers maintain no archive, so a missed day is permanent. The previous design conflated the two, so any fault also cost that day's data.

Phase 4 is unconditional because `ib.fills()` serves the current TWS session only and the daily restart destroys it. A capture running only after a successful trade would lose precisely the fills worth having, which is what happened to 302 executions on 31 July.

6.4 Preflight checks

`ops/preflight.py` runs before every session. Any blocker clears the arm while leaving collection enabled, so a halted book still records.

CHECK	BLOCKS	CATCHES
halt	Yes	Active file in `ops/halts`
costs	Yes	Deployed ticker with no cost entry
cost_drift	Warn	Static cost diverging from the model
data	Yes	Stale prices or NAV
broker	Yes	TWS not listening
margin	Yes	Cushion below 0.10
heartbeat	Warn	Previous session did not run

The cost check exists because `config/costs.yaml` priced 12 tickers while the two live books traded 31. The ledger hard-fails on a missing spread by design, so every ledger update died on the first unknown name, after orders had transmitted. The broker layer caught the exception, printed one line, returned normally, and the run logged "ok".

`DRY_RUN=1` is a human hard halt and always wins. `DRY_RUN=0` means trade if preflight agrees, not trade unconditionally.

6.5 Alerting

`ops/halt.py` escalates across three channels: a file in `ops/halts`, which preflight reads as a hard gate and which survives reboots; a macOS banner and spoken alert; and email. Each channel is wrapped separately so an SMTP timeout cannot prevent a halt being recorded. The file write occurs first and unguarded.

Halts are cleared manually and with attribution:

```
python3 -c "from ops.halt import clear_halt; clear_halt('what was fixed')"
```

Email is not yet configured. It requires a Google App Password, since Gmail rejects account passwords over SMTP. `ALERT_SMTP_USER` and `ALERT_SMTP_PASS` must be added to `config/.env`.

7. Data and external sources

7.1 Sources

SOURCE	CONTENT	COST	SCRIPT
yfinance	CEF prices, NAVs, distributions	Free	`scripts/cef/fetch_daily.py`

SOURCE	CONTENT	COST	SCRIPT
iShares / BlackRock	Daily holdings with per-bond prices	Free	`scripts/holdings/ingest_holdings.py`
State Street, VanEck	Same, other fund families	Free	Same
SEC EDGAR (N-PORT)	Quarterly holdings from 2019	Free	`scripts/nport/`
FINRA	Daily short volume and short interest	Free	`scripts/positioning/`
ICI	Weekly fund flows	Free	`scripts/fetch/fetch_ici_flows.py`
US Treasury	Auction calendar from 1990	Free	`scripts/fetch/build_calendar.py`
IBKR	Historical bid-ask spreads	Entitlement	`scripts/rv/fetch_ibkr_spreads.py`
Cloudflare R2	Our WRDS mirror	Our bucket	`src/data/r2.py`

7.2 The bond price panel

Funds publish a complete daily holdings list including a price for every bond, free and without delay. We had been reading these files to determine holdings and had not recognised them as a bond price source. The union across fifteen funds gives daily prices for 11,423 individual bonds; our best licensed bond dataset at the time was 238 days stale.

Two operational notes. The advertised download link returns a web page rather than data; the working path is `latest-holdings.csv`, not the `.ajax` link the page itself provides. And fund ids must be validated against the published fund name, because two of ours were wrong: we were fetching what we believed was a fallen-angel bond fund and it was the iShares Low Carbon Optimized MSCI ACWI ETF, an equity fund. We caught it only because we print the full column set on first pull and the equity fund had no bond columns. The ingester now rejects any fund whose published name does not match expectation.

No archive exists. Issuers publish the current day only and ignore date parameters, so bond-level backtests are not possible until approximately mid 2027. This is why the collection job must not be skipped.

7.3 Cost model

`config/costs.yaml` holds all trading cost assumptions and prices 45 tickers across four provenance blocks: original tick-floor ETFs, Treasury futures, 11 ETFs with IBKR-measured spreads, and 17 closed-end funds with model-derived spreads.

A July audit found the headline figure had been wrong for the duration of the project. The formula was accurate, matching real measured spreads to within 1 to 2% on seven of eight funds. The sample was the defect.

ERA	COST PER TRADE	COST P.A.
2007–2010	13.91 bp	42.5%
2015–2018	4.49 bp	15.3%
2023–2026	1.73 bp	5.9%

The 21.2% figure was a full-sample average dominated by 2007 to 2014, when these funds were young and thin. Equivalent trading today costs 3.7 times less. We had been charging 2007 costs to modern strategies, imposing a false obstacle on every signal tested. Correcting it rescued nothing, since our failures were absence of edge rather than excessive cost, and it changed no verdict.

8. Open issues

No trading since 1 August. TWS has not been running. Every session since ends `ok_not_armed` with the blocker "nothing listening on 127.0.0.1:7497". Collection, reporting and the watchdog have continued without fault, which is the four-phase design working as intended, but no orders have transmitted since 31 July. Restarting TWS requires an interactive login; automating around that is our first infrastructure priority.

Books over-committed. CEF at $500,000 plus phase 0 at $640,000 is $1.14M claimed against approximately $722,000 of equity, or 158%, at a margin cushion of 0.166. The margin check blocks new exposure below 0.10, but that is a backstop rather than a fix. Resizing is a decision, not a defect.

Two dead scheduled jobs. `com.quantt.book.daily` and `com.quantt.book.weekly` still invoke `/bin/bash` wrappers, which TCC blocks, and reference `ops/books/v2/book_v2_ff.json`, which does not exist. Both should be removed.

Minor. `data/README.md` is stale and states two parquet files are unbuilt when both exist. `boto3` is not installed on the trading machine, so `src/data/r2.py` would fail. `src/analysis/` is empty. No defence exists against distribution cuts (section 3.5), and the frozen spec carries no group exposure limit.

9. Reference

9.1 Documents

DOCUMENT	CONTENT
`HOW_WE_GOT_HERE.md`	Chronological account of the summer
`RESEARCH_AND_METHODOLOGY.md`	Criteria for establishing a result
`RESEARCH_STATE.md`	Live state: deployed, killed, queued
`results/AUDIT_2026-07-31.md`	End-to-end audit
`results/ACADEMIC_REPORT_2026-07-31.md`	Formal write-up
`ops/AUTOMATION.md`	Automation runbook
`CREDIT_RV_PREREG.md` , `E1_PREREG.md`	Pre-registrations for two killed strategies
`results/cef/HOLDOUT_PREREG.md`	Sealed holdout rules, written before opening

9.2 Scripts

SCRIPT	FUNCTION
`scripts/cef/fetch_daily.py`	Daily price and NAV refresh, idempotent
`scripts/cef/validate.py`	Four-test validation battery
`scripts/cef/open_holdout.py`	One-shot sealed holdout opener
`scripts/cef/reconcile_prices.py`	Our prices against the broker's
`scripts/audit/live_pnl_attribution.py`	Live P&L by strategy, reconciled to IBKR
`scripts/audit/cef_factor_audit.py`	Factor exposures and control regressions
`scripts/audit/moc_routing_test.py`	Places one share, cancels, verifies routing
`scripts/holdings/ingest_holdings.py`	Daily bond price collector
`scripts/bench/run_benchmarks.py`	Nine benchmark books, one accounting path
`ops/preflight.py`	The seven safety checks
`ops/capture_fills.py`	Pulls real executions from the broker
`ops/rebuild_ledger.py`	Rebuilds a ledger from broker truth
`docs/build_pdfs.py`	Rebuilds the three team PDFs from markdown

9.3 Literature

Lee, Shleifer and Thaler (1991), "Investor Sentiment and the Closed-End Fund Puzzle", *Journal of Finance* 46(1), and Pontiff (1996), "Costly Arbitrage: Evidence from Closed-End Funds", *Quarterly Journal of Economics* 111(4), on why these discounts exist and persist.

Ellul, Jotikasthira and Lundblad (2011), "Regulatory Pressure and Fire Sales in the Corporate Bond Market", *Journal of Financial Economics* 101(3), the mechanism behind the fallen-angel work in `results/s3/`.

Getmansky, Lo and Makarov (2004), "An Econometric Model of Serial Correlation and Illiquidity in Hedge Fund Returns", *Journal of Financial Economics* 74(3), whose unsmoothing method sits at rank 4 in our research queue.

Bailey and López de Prado (2014), "The Deflated Sharpe Ratio", *Journal of Portfolio Management* 40(5), the source of our deflated Sharpe calculation, and Harvey, Liu and Zhu (2016), "…and the Cross-Section of Expected Returns", *Review of Financial Studies* 29(1), on significance thresholds under multiple testing.

Almgren et al. (2005), "Direct Estimation of Equity Market Impact", *Risk*, the square-root law used in our impact model.

9.4 Maintaining this document

Parts are numbered for reference in messages and pull requests. Any system change requires editing the owning part and adding a changelog row in Part 1. New external sources go in section 7.1; new shared scripts in section 9.2. Items fixed in Part 8 are deleted rather than marked done, so that list always represents the live set of problems.

The PDFs in `docs/pdf/` are build output. The markdown is edited, then `python3 docs/build_pdfs.py` regenerates them and both are committed.